

Logical Operators and Complex Conditions 🤔

Christopher Martin - *Bowdoin College* [🔗](#)
c.martin@bowdoin.edu [✉️](#)

Last class, we saw how we can instruct our computer to ask simple `True` or `False` questions about the state of our programs using the relational operators...

But what if we want to ask more complex questions?



Logical Operators

We can still only ask `True` or `False` questions, but we can combine them to investigate more complex scenarios!

Human Operator	Python Symbol	Example Expression	Explanation
And	<code>and</code>	<code>grade >= 80 and grade < 90</code>	All must be <code>True</code>
Or	<code>or</code>	<code>day == "T" or day == "Th"</code>	At least one must be <code>True</code>
Not	<code>not</code>	<code>not is_snowing</code>	Negates (inverts) the question

Logical Operator in Action

```
1 today: str = "Wednesday"
2 class_cancelled: bool = False
3
4 time: int = 754 # Stores the current time in minutes (754 = 12:34pm)
5
6 is_class_time: bool = time >= 605 and time <= 690 # (605 = 10:05am, 690 = 11:30am)
7 is_class_day: bool = today == "Monday" or today == "Wednesday" or today == "Friday"
8
9 is_during_1101: bool = is_class_time and is_class_day and not class_cancelled
10
11 print(f"Today is {today} at {time // 60}:{time % 60}")
12 if is_during_1101:
13     print("I should be in CSCI: 1101! 😊")
14 else:
15     print("No CSCI: 1101! 😭")
```

Today is Wednesday at 12:34

No CSCI: 1101! 😭

Mutually Exclusive Situations

Last class we saw how we can conditionally execute code...

```
1 temperature: float = 31.5
2
3 if temperature < 32.0:
4     print("It is freezing! 🥶")
5     print("Maybe it will snow! 🧊")
6 else:
7     print("It is above freezing! ☀️")
8     print("Maybe it will rain! ☔")
```

It is freezing! 🥶
Maybe it will snow! 🧊

If-else statements work great for representing binary conditions (situations where there are two mutually exclusive outcomes) but what if we need more?

Mutually Exclusive Situations

Let's look at a situation where we have three mutually exclusive outcomes!

```
1 temperature: float = 45.0
2
3 if temperature < 32.0:
4     print("It's freezing! 🥶")
5 else:
6     if temperature < 50.0:
7         print("It's chilly. Bundle up! 🧥")
8     else:
9         print("It's warm and sunny! 😎")
10
11 # Or, we could do it like this?
12 if temperature < 32.0:
13     print("It's freezing! 🥶")
14 if temperature >= 32.0 and temperature < 50.0:
15     print("It's chilly. Bundle up! 🧥")
16 if temperature >= 50.0:
17     print("It's warm and sunny! 😎")
```

It's chilly. Bundle up! 🧥

The `elif` Clause

The `elif` clause, short for "else if", lets you test additional backup conditions or other multiple mutually exclusive situations tied to a single `if` statement!

If no condition evaluates to `True`, then the `else` clause runs if there is one.

```
1 temperature: float = 45.0
2
3 if temperature < 32.0:
4     print("It's freezing! 🥶")
5 elif temperature < 50.0:
6     print("It's chilly. Bundle up! 🧥")
7 else:
8     print("It's warm and sunny! 😎")
```

It's chilly. Bundle up! 🧥

The `elif` Clause in Action

You can have as many `elif` clauses as needed!

```
1 score: float = 87.5
2
3 if score >= 90:
4     print("You got an A!")
5 elif score >= 80:
6     print("You got a B!")
7 elif score >= 70:
8     print("You got a C!")
9 elif score >= 60:
10    print("You got a D!")
11 else:
12    print("You got an F!")
```

You got a B!

The `elif` Clause - Statement Order

Python evaluates each condition in order until one is found to be `True`.

Only that condition's code will be executed - the rest will be skipped!



```
1 score: float = 87.5
2
3 # Ordering the statements wrong can result in incorrect behavior!
4 if score >= 60:
5     print("You got a D!")
6 elif score >= 90:
7     print("You got an A!")
8 elif score >= 70:
9     print("You got a C!")
10 elif score >= 80:
11     print("You got a B!")
12 else:
13     print("You got an F!")
```

The `len()` Function

The `len()` function returns the number of characters in a collection.

We commonly use it to check how many characters there are in a string.

This includes any letters, numbers, whitespace, and punctuation.

```
1 name: str = "Christopher"
2
3 length: int = len(name)
4
5 print(f"The string '{name}' has {length} characters.")
```

The `in` Operator

The `in` operator checks if a collection contains a specified element!

We commonly use it to check if a string contains some other specified text.

It evaluates to `True` if the value of the left-operand is contained within the right-operand. otherwise, it evaluates to `False`.

```
1 # Try with other words / sentences!
2 message: str = "Team"
3 substring: str = "i"
4
5 if substring in message:
6     print(f"{message}\" contains \"{substring}\"!")
7 else:
8     print(f"There is no \"{substring}\" in \"{message}\"!")
```

More Questions we can ask about String data

Strings include many helpful tools called that can inspect and modify their data!

We will discuss these **methods** more next week, but for right now let's look at some more of the additional ways we can ask questions about the contents of a string!

Method	Description
<code>my_str.islower()</code>	Returns <code>True</code> if all letters in a given <code>my_str</code> are lowercase
<code>my_str.isupper()</code>	Returns <code>True</code> if all letters in a given <code>my_str</code> are uppercase
<code>my_str.isdigit()</code>	Returns <code>True</code> if all characters in a given <code>my_str</code> are digits
<code>my_str.startswith(sub_str)</code>	Returns <code>True</code> if a given <code>my_str</code> starts with the specified <code>sub_str</code>
<code>my_str.endswith(sub_str)</code>	Returns <code>True</code> if a given <code>my_str</code> ends with the specified <code>sub_str</code>